

UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO
FACULTAD DE ESTUDIOS SUPERIORES ACATLÁN
Licenciatura en Matemáticas Aplicadas y Computación

MANUAL TÉCNICO

Herramienta de Procesos Markovianos de Decisión (MDP)

Materia: Procesos Estocásticos
Profesora: Cuéllar Aguayo Ada Ruth

Integrantes:
Hernández Pérez Victoria
Martínez Macouzet Enrique

México, 2026

1. Introducción

Este documento es el manual técnico completo de la herramienta MDP, un programa escrito en lenguaje C que implementa cinco métodos distintos para resolver Procesos Markovianos de Decisión. El programa se desarrolló como proyecto final de la materia de Procesos Estocásticos.

El objetivo de este manual no es solo describir qué hace cada función, sino explicar el por qué de cada decisión de diseño: por qué se eligió Gauss-Jordan en lugar de Gauss simple, por qué el Simplex usa dos fases en lugar de la Gran M, por qué ciertos arreglos son globales en lugar de locales, y qué sucede exactamente en cada iteración de los métodos iterativos. Al terminar de leer este manual, cualquier persona que conozca C y álgebra lineal básica debe poder reproducir el programa desde cero.

1.1 ¿Qué es un Proceso Markoviano de Decisión?

Un Proceso Markoviano de Decisión (MDP) es un modelo matemático para la toma de decisiones en situaciones donde los resultados son parcialmente aleatorios y parcialmente controlados por un agente decisor. La propiedad fundamental que lo define es la **propiedad de Markov**: el futuro del sistema depende únicamente del estado presente, no de la historia pasada.

Formalmente, un MDP se define por la 5-tupla (S, D, P, C, π) :

Componente	Notación	Descripción
Conjunto de estados	$S = \{s_0, s_1, \dots, s_{n-1}\}$	Todas las situaciones posibles del sistema
Conjunto de decisiones	$D = \{d_0, d_1, \dots, d_{k-1}\}$	Acciones disponibles para el decisor
Probabilidades de transición	$P(j i, d)$	Probabilidad de ir al estado j desde el estado i tomando la decisión d
Costos o ganancias	$C(i, d)$	Costo o ganancia inmediata al estar en el estado i y tomar la decisión d
Política	$\pi : S \rightarrow D$	Función que asigna una decisión a cada estado; la incógnita que se optimiza

El objetivo es encontrar la política óptima π^* que minimice el costo total esperado a largo plazo (problema de costos, tipo 0), o que maximice la ganancia total esperada (problema de ganancias, tipo 1).

1.2 ¿Por qué cinco métodos?

No existe un único algoritmo universalmente superior para todos los MDP. Los cinco métodos implementados abordan el problema desde perspectivas distintas:

Dos son **exactos**: Enumeración Exhaustiva evalúa todas las combinaciones posibles de decisiones; Programación Lineal convierte el problema combinatorio en un problema continuo convexo con garantía de óptimo global.

Tres son **iterativos**: Mejoramiento de Políticas (Howard sin descuento), Mejoramiento con Descuento y Aproximaciones Sucesivas (Value Iteration). Los tres convergen al óptimo pero por caminos matemáticos distintos, lo que permite verificar por coincidencia de resultados y estudiar las diferencias en los vectores de valor V .

La herramienta permite ejecutar todos ellos sobre el mismo modelo y comparar directamente sus resultados en una tabla resumen.

2. Arquitectura del Programa

2.1 Compilación y Ejecución

El programa se compila con GCC. El flag `-lm` es obligatorio porque el código utiliza funciones de la biblioteca matemática estándar (`fabs`, `round`, `isnan` de `<math.h>`):

```
gcc -o mdp mdp.c -lm
./mdp
```

Si se omite `-lm`, el enlazador reportará errores del tipo `undefined reference to `fabs'`. Esto ocurre porque en Linux, las funciones matemáticas no están en `libc` sino en una biblioteca separada que hay que enlazar explícitamente.

2.2 Estructura General del Archivo Fuente

El archivo `mdp.c` está organizado secuencialmente. Cada sección cumple una función única y las secciones posteriores dependen de las anteriores:

Sección	Contenido principal
Includes y constantes	<code>#include</code> de cabeceras estándar; <code>#define</code> de límites máximos
Estructuras de datos	<code>typedef struct</code> <code>ModeloMDP</code> y variables globales
Funciones utilitarias	<code>limpiar()</code> , <code>pausar()</code> , <code>leer_numero()</code> , <code>leer_entero_rango()</code> , <code>fmt_d()</code>
Portada	<code>mostrar_portada()</code> con arte ASCII y colores ANSI
Ingreso de datos	<code>ingresar_datos()</code> con validaciones y verificación interactiva de costos
Visualización	<code>visualizar()</code> : tablas de costos/ganancias y matrices de transición por decisión y por política
Generación de políticas	<code>generar_politicas()</code> y <code>buscar_indice_politica()</code>
Gauss-Jordan	Eliminación gaussiana con pivoteo parcial; base numérica de varios métodos
Enumeración Exhaustiva	Evalúa todas las políticas; detecta empates; muestra ranking
Simplex de Dos Fases	Tabla iterativa del Simplex con variables artificiales
Programación Lineal	Formulación del PPL, solución y detección de políticas alternativas

Mejoramiento de Políticas	Método iterativo de Howard sin descuento
Mejoramiento con Descuento	Método iterativo de Howard con factor $\alpha \in (0,1)$
Aproximaciones Sucesivas	Value iteration con ecuaciones expandidas por iteración
Comparación de Métodos	Ejecuta los 5 métodos en silencio; tabla resumen; reporta políticas alternativas
Despedida	Animación ASCII de fuegos artificiales en terminal
Guardar / Cargar / Reporte	E/S de archivos .mdp (texto plano) y .txt (reporte legible)
Menú Principal (main)	Bucle do-while con guards de validación de modelo cargado

2.3 Constantes Globales

Todas las constantes se definen con `#define` al inicio del archivo. Usar constantes simbólicas en lugar de números literales tiene dos ventajas: el código es auto-documentado (`MAX_ESTADOS` es más claro que `20`) y cambiar un límite requiere modificar un solo lugar.

Constante	Valor	Propósito
<code>MAX_ESTADOS</code>	20	Número máximo de estados que puede tener un modelo
<code>MAX_DECISIONES</code>	10	Número máximo de decisiones disponibles
<code>MAX_NOMBRE</code>	16	Longitud máxima (en caracteres) del nombre de un estado o decisión
<code>MAX_VARS</code>	200	Máximo de variables para el Simplex (= <code>MAX_ESTADOS</code> × <code>MAX_DECISIONES</code>)
<code>MAX_RESTRICCIONES</code>	22	Máximo de restricciones para el Simplex (= <code>MAX_ESTADOS</code> + 2)
<code>MAX_POLITICAS</code>	100 000	Tope de políticas para la enumeración exhaustiva y buffers globales
<code>MAX_ITER_SIMPLEX</code>	200	Iteraciones máximas del Simplex antes de abortar para evitar ciclos infinitos

2.4 Estructura Principal: ModeloMDP

Todo el modelo matemático vive en un único struct global. **No se usa asignación dinámica de memoria**; todos los arreglos son estáticos con los tamaños máximos definidos arriba. Esta decisión simplifica el código: no hay malloc/free, no hay posibilidad de memory leaks y el tamaño máximo de memoria del programa es predecible en tiempo de compilación.

```
typedef struct {
    int  num_estados;           // n: cantidad de estados
    int  num_decisiones;        // k: cantidad de decisiones
    int  tipo;                  // 0 = minimizar costos, 1 = maximizar ganancias

    char estados[MAX_ESTADOS][MAX_NOMBRE];
    char decisiones[MAX_DECISIONES][MAX_NOMBRE];

    double costos[MAX_ESTADOS][MAX_DECISIONES];
    // C[i][d] = costo/ganancia en estado i tomando decisión d

    double transiciones[MAX_DECISIONES][MAX_ESTADOS][MAX_ESTADOS];
    // P[d][i][j] = P(j | i, d)
    // Orden: PRIMERO la decisión, LUEGO el estado origen, LUEGO el destino

    bool estados_afectados[MAX_DECISIONES][MAX_ESTADOS];
    // true si la decisión d aplica al estado i
} ModeloMDP;

ModeloMDP m; // instancia global única
bool modelo_cargado = false; // bandera de validación
double alpha_descuento = 0.9; //  $\alpha$  para Mejoramiento con Descuento
double alpha_aproximacion = 1.0; //  $\alpha$  para Aproximaciones Sucesivas
double epsilon_aproximacion = 0.001; //  $\epsilon$  de convergencia
```

NOTA SOBRE EL ORDEN DEL TENSOR `transiciones[d][i][j]`: se indexa primero por decisión *d*, luego por estado origen *i* y finalmente por estado destino *j*. Este orden no es arbitrario: al fijar la política (es decir, al saber qué decisión *d_i* tomar en cada estado *i*), los bucles acceden a `transiciones[di][i][j]` con *d_i* e *i* fijos y *j* variando. Este patrón de acceso secuencial es el más eficiente en C, donde los arreglos se almacenan en orden row-major (fila por fila).

Variables globales de buffers grandes

Además del struct `m`, hay tres arreglos declarados `static` a nivel de archivo:

```
static int g_politicas[MAX_POLITICAS][MAX_ESTADOS];
static int g_optimas_pol[MAX_POLITICAS][MAX_ESTADOS];
static int g_optimas_idx[MAX_POLITICAS];
```

¿Por qué globales y no locales? Con `MAX_POLITICAS=100 000` y `MAX_ESTADOS=20`, cada arreglo bidimensional ocupa $100\,000 \times 20 \times 4$ bytes = **8 MB**. El stack del proceso en Linux tiene por defecto entre 1 y 8 MB. Declarar estos arreglos como variables locales causaría un desbordamiento de stack (stack overflow) silencioso o un segfault. Al declararlos `static` en el alcance del archivo, el compilador los coloca en el segmento BSS/data del ejecutable, que no tiene esa restricción.

3. Funciones Utilitarias

Estas funciones son la base de toda la interacción con el usuario. Se definen antes que cualquier otra función del programa porque C requiere que las funciones estén declaradas antes de ser usadas (o tener un forward declaration).

3.1 leer_numero()

Función robusta de lectura de valores reales desde stdin. Acepta tanto números decimales como fracciones del tipo a/b. Esto es esencial para ingresar probabilidades exactas (como $1/3 = 0.333\dots$) sin error de redondeo manual.

```
double leer_numero() {
    char buf[64];
    while(1) {
        if(!fgets(buf, sizeof(buf), stdin)) { buf[0]='\0'; buf[1]='\0'; }
        buf[strcspn(buf, "\n")] = 0; // eliminar el '\n' final
        if(buf[0] == '\0') { printf(" (vacío, 0): "); continue; }

        char *p = strchr(buf, '/'); // detectar fracción tipo '1/3'
        if(p) {
            *p = 0;
            double n = atof(buf), d = atof(p+1);
            if(fabs(d) < 1e-9) { printf(" División entre cero."); continue; }
            return n/d;
        }
        char *end;
        double val = strtod(buf, &end); // más robusto que atof
        if(end == buf) { printf(" Entrada inválida."); continue; }
        return val;
    }
}
```

¿Por qué strtod en lugar de atof? La función atof es un envoltorio de strtod que ignora si la conversión fue exitosa. strtod expone el puntero end: si `end == buf`, significa que no se pudo leer ningún dígito (la cadena no era un número). Esto permite detectar entradas como "abc" que atof convertiría silenciosamente a 0.0.

¿Por qué aceptar fracciones tipo 1/3? En probabilidades exactas como $1/3$, si el usuario escribe 0.333 o incluso 0.3333, la suma de la fila nunca llegará exactamente a 1.0 aunque matemáticamente debería. Escribir $1/3$ produce el valor de punto flotante más cercano a $1/3$ que ofrece el tipo double, que es el mismo que usarían los cálculos internos al operar con esas probabilidades.

3.2 leer_entero_rango(min, max)

Valida que la entrada sea un entero dentro del rango [min, max]. Usa `strtol()` en lugar de `atoi()` porque `strtol()` permite detectar si la cadena contenía caracteres no numéricos (`end != buf`). El bucle es infinito intencionalmente: la función no regresa hasta obtener un valor válido, lo que garantiza que el código que la llama nunca recibe un valor fuera del rango esperado.

3.3 `fmt_d(v)` — Formato Numérico Inteligente

Devuelve el número `v` sin decimales si es entero (por ejemplo 5 en lugar de 5.0000), o con hasta 4 cifras significativas si no lo es. Usa un buffer estático rotativo de 8 posiciones para poder usarse en varios `printf()` consecutivos sin pisarse.

```
static char _fmt_bufs[8][32];
static int _fmt_idx = 0;

static const char *fmt_d(double v) {
    char *buf = _fmt_bufs[_fmt_idx++ % 8]; // buffer circular de 8 slots
    if(fabs(v - round(v)) < 1e-9)
        snprintf(buf, 32, "%.0f", round(v)); // entero: sin decimales
    else
        snprintf(buf, 32, "%.4g", v);          // real: 4 cifras significativas
    return buf;
}
```

¿Por qué el buffer rotativo? Si se usara un solo buffer estático, llamar `printf("%s y %s", fmt_d(a), fmt_d(b))` produciría resultados incorrectos: la segunda llamada a `fmt_d` sobrescribiría el buffer antes de que `printf` leyera el primer valor. Con 8 buffers rotativos, hasta 8 llamadas consecutivas en el mismo `printf` son seguras.

3.4 `limpiar()` y `pausar()`

`limpiar()` envía la secuencia de escape ANSI `\033[2J\033[H` que limpia la pantalla y reposiciona el cursor en la esquina superior izquierda.

`pausar()` consume todos los caracteres hasta el siguiente salto de línea, logrando el efecto de "presiona Enter para continuar" sin dejar el buffer de entrada contaminado para la siguiente lectura. El **buffer contamination** es un problema común en C: si un `scanf` o `getchar` previo no consumió el `\n`, la siguiente lectura con `fgets` lo recibe vacío instantáneamente.

4. Ingreso de Datos — `ingresar_datos()`

Esta función guía al usuario paso a paso para construir el ModeloMDP completo. El flujo de ejecución es secuencial: primero el tipo y dimensiones del modelo, luego los nombres, luego los costos y finalmente las matrices de transición.

Paso	Datos que se capturan	Función de lectura
1	Tipo de modelo: 0 = costos (minimizar), 1 = ganancias (maximizar)	<code>leer_entero_rango(0, 1)</code>
2	Número de estados $n \in [1, \text{MAX_ESTADOS}]$	<code>leer_entero_rango(1, 20)</code>
3	Nombre de cada estado (hasta 15 caracteres)	<code>fgets + trim del \n</code>
4	Número de decisiones $k \in [1, \text{MAX_DECISIONES}]$	<code>leer_entero_rango(1, 10)</code>
5	Nombre de cada decisión	<code>fgets + trim</code>
6	¿La decisión d aplica al estado i ? (por cada par d, i)	<code>leer_entero_rango(0, 1)</code>
7	Costo/ganancia $C(i, d)$ para cada estado afectado	<code>leer_numero()</code>
8	Verificación interactiva de costos/ganancias ingresados	<code>leer_entero_rango(0, 1)</code> en bucle
9	Matriz de transición $P[d][i][j]$ para cada estado afectado i	<code>leer_numero()</code> con validación de suma = 1

4.1 ¿Qué significa "no toda decisión aplica a todo estado"?

La matriz `estados_afectados[d][s]` permite modelar situaciones reales donde ciertas acciones son físicamente imposibles o no tienen sentido en determinados estados. Por ejemplo: en un modelo de mantenimiento de una máquina, la decisión "Reparación mayor" podría no aplicar a la máquina cuando está en estado "Nueva". Si `estados_afectados[d][s] = false`, el costo `costos[s][d]` y la fila de transición `transiciones[d][s][*]` no se ingresan ni se usan en ningún cálculo.

4.2 Verificación Interactiva de Costos

Después de ingresar todos los costos/ganancias de una decisión, el programa despliega un resumen numerado y solicita confirmación antes de continuar. Si el usuario detecta un error tipográfico, puede corregir un valor a la vez sin reingresar todo el modelo:

```
int confirmado = 0;
while(!confirmado) {
    // 1. Construir lista de estados afectados
```

```

int num_af = 0, af_idx[MAX_ESTADOS];
for(int s = 0; s < m.num_estados; s++)
    if(m.estados_afectados[d][s]) af_idx[num_af++] = s;

// 2. Mostrar resumen numerado
printf("Resumen de costos para '%s':\n", m.decisiones[d]);
for(int k = 0; k < num_af; k++)
    printf("    [%d] %-10s : %s\n", k+1, m.estados[af_idx[k]],
        fmt_d(m.costos[af_idx[k]][d]));

// 3. Confirmación
printf("¿Correctos? (1=Sí / 0=No): ");
if(Leer_entero_rango(0,1)) { confirmado = 1; }
else {
    printf("¿Cuál corregir? (1-%d): ", num_af);
    int cual = Leer_entero_rango(1, num_af);
    printf("Nuevo costo en %s: ", m.estados[af_idx[cual-1]]);
    m.costos[af_idx[cual-1]][d] = Leer_numero();
    // El bucle vuelve y muestra el resumen actualizado
}
}

```

4.3 Validación de Matrices de Transición

Cada fila $P[d][i][*]$ se valida en dos aspectos: rango de cada probabilidad individual $\in [0,1]$ y suma total de la fila = 1.0 (con tolerancia 0.001). Si alguna condición falla se ofrece reingresar la fila completa hasta 3 intentos.

¿Por qué goto? El uso de goto `retry_trans` está deliberadamente restringido a este contexto de reintento de fila. Es el patrón más claro y directo para "volver al inicio del bloque de lectura": una alternativa con `do-while` requeriría una bandera adicional y haría el código más largo sin ningún beneficio. C11 no tiene `redo` como sí lo tiene Perl; goto es la herramienta correcta aquí.

¿Por qué tolerancia 0.001 y no exactamente 0? Las operaciones de punto flotante introducen errores de redondeo. Si el usuario ingresa 0.3, 0.3 y 0.4, la suma en `double` puede ser 0.9999999999999999 o 1.0000000000000002 dependiendo del procesador. Exigir `suma == 1.0` exactamente rechazaría entradas perfectamente válidas. Una tolerancia de 0.001 es amplia para el redondeo del punto flotante pero suficientemente estricta para detectar errores reales como olvidar un estado.

5. Generación de Políticas

5.1 Definición Formal

Una política π es una función $\pi : S \rightarrow D$ que asigna a cada estado una decisión. Dado que no toda decisión aplica a todos los estados, el conjunto de decisiones disponibles por estado puede variar. Si el estado i tiene m_i decisiones disponibles, el número total de políticas distintas es:

$$|\Pi| = m_0 \times m_1 \times m_2 \times \dots \times m_{n-1}$$

Por ejemplo: si hay 3 estados y cada uno tiene 2 decisiones disponibles, existen $2^3 = 8$ políticas. Con 3 estados y 3 decisiones por estado serían $3^3 = 27$ políticas. El tope `MAX_POLITICAS = 100 000` limita la combinatoria para garantizar que el programa termine en tiempo razonable.

5.2 Algoritmo: Conteo en Base Mixta

`generar_politicas()` enumera todas las combinaciones usando el mismo principio que contar en una base numérica donde cada posición puede tener una base diferente. El índice de política p se decodifica de derecha a izquierda con divisiones sucesivas:

```
int generar_politicas(int politicas[][MAX_ESTADOS]) {
    // dec_por_estado[i][k] = k-ésima decisión válida del estado i
    int dec_por_estado[MAX_ESTADOS][MAX_DECISIONES];
    int max_op[MAX_ESTADOS] = {0};
    for(int i = 0; i < m.num_estados; i++)
        for(int d = 0; d < m.num_decisiones; d++)
            if(m.estados_afectados[d][i])
                dec_por_estado[i][max_op[i]++] = d;

    int total = 1;
    for(int i = 0; i < m.num_estados; i++) total *= max_op[i];

    // Decodificar p en base mixta: de derecha a izquierda
    for(int p = 0; p < total; p++) {
        int temp = p;
        for(int i = m.num_estados-1; i >= 0; i--) {
            politicas[p][i] = dec_por_estado[i][temp % max_op[i]];
            temp /= max_op[i];
        }
    }
    return total;
}
```

ANALOGÍA: Funciona exactamente como contar horas:minutos:segundos, donde cada posición tiene una base diferente (24, 60, 60). Con 3 estados y opciones (3, 2, 4), la política número $p=5$ se decodifica así: dígito 2 (base 4, estado más derecho): $5 \% 4 = 1 \rightarrow$ segunda decisión del estado 2 $\text{temp} = 5 / 4 = 1$ dígito 1 (base 2): $1 \% 2 = 1 \rightarrow$ segunda decisión del estado 1 $\text{temp} = 1 / 2 = 0$ dígito 0 (base 3): $0 \% 3 = 0 \rightarrow$ primera decisión del estado 0 Resultado: política (d_0, d_1, d_2)

5.3 buscar_indice_politica()

Función auxiliar que recorre el arreglo de políticas generadas y devuelve el índice (base 1) de la primera que coincide exactamente con el arreglo buscado. Se usa para asignar el número R_k a la política óptima en cada método, garantizando que los números sean comparables entre métodos.

6. Eliminación de Gauss-Jordan con Pivoteo Parcial

La eliminación de Gauss-Jordan es el núcleo numérico del programa. Es usada directamente por Enumeración Exhaustiva, Mejoramiento de Políticas y Mejoramiento con Descuento para resolver los sistemas de ecuaciones lineales que surgen al evaluar una política fija.

6.1 Firma de la Función

```
void gauss_jordan(
    double a[MAX_ESTADOS+1][MAX_ESTADOS+2], // matriz aumentada [A | b]
    int n, // número de ecuaciones / incógnitas
    int mostrar, // 1 = imprime cada paso, 0 = silencioso
    const char **nombres_cols // etiquetas de columna (puede ser NULL)
);
```

6.2 Algoritmo Detallado Paso a Paso

Paso 1 — Pivoteo parcial: Para la columna (variable) k , se busca el renglón $r \geq k$ con el mayor valor absoluto $a[r][k]$. Se intercambian los renglones k y r . **¿Por qué es necesario?** Si el elemento diagonal $a[k][k]$ es muy pequeño (cercano a cero), dividir entre él amplifica los errores de redondeo en todas las operaciones siguientes. El pivoteo parcial garantiza que siempre se divide entre el mayor valor disponible en esa columna.

Paso 2 — División del pivote: Se divide toda la fila k entre $a[k][k]$, obteniendo 1 en la posición diagonal (k,k) .

Paso 3 — Eliminación completa: Para cada renglón $i \neq k$, se calcula el factor $f = a[i][k]$ y se actualiza $a[i][j] -= f * a[k][j]$ para todo j . Esto deja ceros en **toda** la columna k , incluyendo los renglones superiores ($i < k$).

```
// Pivoteo parcial
int piv = col;
for(int row = col+1; row < n; row++)
    if(fabs(a[row][col]) > fabs(a[piv][col])) piv = row;
// Intercambio de renglones
for(int j = 0; j <= n; j++) { double t = a[col][j]; a[col][j] = a[piv][j]; a[piv][j] = t; }

// División del pivote
double div = a[col][col];
if(fabs(div) < 1e-12) return; // sistema singular, sin solución
for(int j = 0; j <= n; j++) a[col][j] /= div;

// Eliminación en AMBAS direcciones (Gauss-Jordan completo)
for(int row = 0; row < n; row++) {
    if(row == col) continue;
    double f = a[row][col];
    for(int j = 0; j <= n; j++) a[row][j] -= f * a[col][j];
}
```

¿POR QUÉ GAUSS-JORDAN Y NO GAUSS SIMPLE? Gauss simple sólo elimina hacia abajo (triangulación) y requiere un paso adicional de sustitución hacia atrás para obtener la

solución. Gauss-Jordan elimina en ambas direcciones, obteniendo la forma identidad reducida directamente. La solución se lee de la última columna: $x_i = a[i][n]$. Esto simplifica el código al hacer que la función sea autocontenida: quien la llama sólo pasa la matriz aumentada y lee la solución en esa misma columna al terminar, sin código adicional de sustitución.

6.3 Casos en que Gauss-Jordan se llama

Contexto de llamada	Sistema que resuelve	Parámetro mostrar
Enumeración Exhaustiva	$\pi P = \pi$ con condición $\sum \pi = 1$ (distribución estacionaria)	1 (muestra pasos)
Mejoramiento de Políticas	Sistema de Howard: $g + V = C + PV$	1 (muestra pasos)
Mejoramiento con Descuento	$(I - \alpha P)V = c$	1 (muestra pasos)
Comparación (silencioso)	Todos los anteriores, sin output	0 (silencioso)
Búsqueda de alternativas PL	Sistema de Howard para verificar g de cada política	0 (silencioso)

7. Simplex de Dos Fases

El método Simplex de Dos Fases resuelve el Problema de Programación Lineal (PPL) del MDP. Las restricciones son igualdades exactas ($Ax = b$), lo que impide construir una Base Factible Inicial con variables de holgura estándar.

7.1 ¿Por qué Dos Fases y no el Método de la Gran M?

El Método de la Gran M añade un coeficiente enorme M a las variables artificiales en la función objetivo original. El problema práctico es que con $M = 1\,000\,000$ y costos del modelo en el orden de las centenas, las operaciones de punto flotante introducen errores que pueden distorsionar el pivoteo. En pruebas internas el Simplex de Dos Fases produce resultados más estables.

Las Dos Fases evitan este problema completamente: la Fase 1 minimiza la suma de artificiales con coeficientes exactamente 1 (sin M), y la Fase 2 trabaja con el objetivo real sin artificiales en la función.

7.2 Fase 1: Encontrar una Base Factible Real

Se introduce una variable artificial $a_i \geq 0$ en cada restricción i. La Fase 1 resuelve:

$$\text{Min } W = \sum_i a_i$$

Si $W^* = 0$ al terminar, todas las artificiales salieron de la base y se obtuvo una BFS real del problema original. Si $W^* > 0$, el problema no tiene solución factible (las restricciones son inconsistentes).

```
// Construcción de la tabla inicial de la Fase 1
// Columnas: n_vars variables reales | n_restr artificiales | Sol
for(int i = 0; i < n_restr; i++) {
    for(int j = 0; j < n_vars; j++) tabla[i][j] = A_eq[i][j];
    tabla[i][n_vars + i] = 1.0; // variable artificial i en columna n_vars+i
    tabla[i][n_total] = b_eq[i];
}
// Fila Z para Fase 1: minimizar suma de artificiales
for(int j = n_vars; j < n_total; j++) tabla[n_restr][j] = 1.0;
// Ajuste: restar las filas de restricción para que la fila Z sea consistente
for(int i = 0; i < n_restr; i++)
    for(int j = 0; j <= n_total; j++) tabla[n_restr][j] -= tabla[i][j];
```

7.3 Fase 2: Optimizar el Objetivo Real

Se reemplaza la función objetivo por la del MDP. El programa realiza internamente siempre una minimización; en problemas de maximización los coeficientes se niegan antes de pasarlos:

```
// Para problemas de costos: c_obj[v] = costo(estado_v, decision_v)
// Para problemas de ganancias: c_obj[v] = -ganancia(...) (se niega)
for(int i = 0; i < m.num_estados; i++)
    for(int d = 0; d < m.num_decisiones; d++)
        if(var_idx[i][d] >= 0)
            c_obj[var_idx[i][d]] = (m.tipo==0) ? m.costos[i][d] : -m.costos[i][d];

// Al imprimir, se muestran los coeficientes REALES (sin negar):
printf("%s Z = ...", m.tipo==0 ? "Min" : "Max");
```


7.4 Regla de Entrada y Salida (Criterio de Dantzig)

Variable que entra (columna pivote): la de coeficiente más negativo en la fila Z. Si no hay coeficientes negativos, se llegó al óptimo (todos los movimientos posibles empeoran Z).

Variable que sale (renglón pivote): la que produce la menor razón positiva $Sol[i] / a[i][col_piv]$. Esta regla garantiza que la solución se mantenga no-negativa (factible) después del pivote.

Límite de iteraciones: $MAX_ITER_SIMPLEX = 200$ previene ciclos infinitos (posibles cuando hay degeneración, es decir, cuando alguna variable básica vale 0). En la práctica el Simplex converge en muy pocas iteraciones para los modelos que maneja este programa.

8. Método 1: Enumeración Exhaustiva

Evalúa sistemáticamente todas las políticas posibles. Para cada política π fija resuelve el sistema de estado estacionario $\pi P = \pi$ con la condición de normalización $\sum \pi_i = 1$, y calcula el valor esperado $g = \sum \pi_i \cdot C(i, \pi(i))$.

8.1 Sistema de Ecuaciones de Estado Estacionario

Para una política fija, la distribución estacionaria π satisface $\pi P = \pi$. Reescribiendo componente a componente: $\pi(j) = \sum_i \pi(i) \cdot P(i,j | \pi(i))$ para todo j . Reorganizando:

$$\pi(j) - \sum_i \pi(i) \cdot P(i,j) = 0$$

En el código se construye la matriz aumentada A de tamaño $n \times (n+1)$. Las primeras $n-1$ filas son las ecuaciones de balance (se omite la última porque es linealmente dependiente de las demás), y la fila $n-1$ es la condición de normalización:

```
// Ecuaciones de balance (filas 0 a n-2)
for(int j = 0; j < n-1; j++) {
    for(int i = 0; i < n; i++)
        A[j][i] = (i==j) ? 1.0 - P[i][j] : -P[i][j];
    A[j][n] = 0.0;
}
// Condición de normalización (última fila):  $\sum \pi_i = 1$ 
for(int i = 0; i < n; i++) A[n-1][i] = 1.0;
A[n-1][n] = 1.0;

gauss_jordan(A, n, mostrar, nombres);

// Leer solución:  $\pi_i = A[i][n]$ 
// Calcular costo/ganancia esperado por etapa
double g = 0;
for(int i = 0; i < n; i++) g += A[i][n] * costos[i];
```

¿POR QUÉ OMITIR LA ÚLTIMA ECUACIÓN DE BALANCE? Para cualquier matriz estocástica P , la suma de todas las ecuaciones de balance da $0 = 0$ (identidad), lo que significa que una de las ecuaciones es siempre linealmente dependiente de las demás. Si se incluyeran las n ecuaciones de balance y la normalización, el sistema tendría $n+1$ ecuaciones con n incógnitas y el sistema de Gauss-Jordan podría detectarlo como singular. La solución correcta es incluir solo $n-1$ ecuaciones de balance y reemplazar la última con la condición de normalización $\sum \pi_i = 1$, que garantiza una solución única.

8.2 Detección y Reporte de Empates

El programa mantiene una lista de políticas óptimas empatadas. Si una nueva política tiene el mismo valor g^* que la mejor actual (diferencia $< 1e-9$), se añade a la lista en lugar de reemplazarla:

```
if ((tipo==0 && g < mejor_g - 1e-9) || (tipo==1 && g > mejor_g + 1e-9)) {
    // Nueva mejor: reiniciar lista
    mejor_g = g; num_optimas = 0;
    g_optimas_idx[0] = p+1;
    memcpy(g_optimas_pol[0], pol, n*sizeof(int));
}
```

```
    num_optimas = 1;
    printf("  <- NUEVO ÓPTIMO");
} else if(fabs(g - mejor_g) < 1e-9) {
    // Empate: agregar a la lista
    g_optimas_idx[num_optimas] = p+1;
    memcpy(g_optimas_pol[num_optimas++], pol, n*sizeof(int));
    printf("  <- EMPATE ÓPTIMO");
}
```

8.3 Complejidad Computacional

La complejidad es $O(|\Pi| \times n^3)$, donde $|\Pi|$ es el número total de políticas y n^3 proviene del costo de Gauss-Jordan. Con 5 estados y 3 decisiones por estado habría 243 políticas; con 7 estados y 4 decisiones serían 16 384 políticas. MAX_POLITICAS = 100 000 limita el recorrido para modelos extremadamente grandes.

9. Método 2: Programación Lineal

Reformula el MDP como un Problema de Programación Lineal (PPL). Las variables de decisión $Y_{i,d}$ representan la frecuencia estacionaria de visitar el estado i tomando la decisión d en el largo plazo. Este enfoque convierte el problema combinatorio de buscar la política óptima en un problema de optimización continua convexo.

9.1 Formulación Matemática Completa

Sea $Y_{i,d}$ la frecuencia estacionaria de estar en el estado i y tomar la decisión d . El PPL es:

Función objetivo:

$$\text{Max } Z = \sum_i \sum_d C(i,d) \cdot Y(i,d) \quad (\text{para ganancias})$$

$$\text{Min } Z = \sum_i \sum_d C(i,d) \cdot Y(i,d) \quad (\text{para costos})$$

Restricciones de balance de flujo (para cada estado destino $s = 0, \dots, n-2$):

$$\sum_d Y(s,d) - \sum_i \sum_d P(i \rightarrow s, d) \cdot Y(i,d) = 0$$

Restricción de normalización:

$$\sum_i \sum_d Y(i,d) = 1$$

No negatividad: $Y(i,d) \geq 0$ para todo i, d válido

La n -ésima ecuación de balance se omite por ser linealmente dependiente (igual que en Enumeración Exhaustiva), dejando n restricciones en total: una de normalización y $n-1$ de balance.

9.2 Construcción del Sistema en el Código

El código asigna un índice global a cada par (i, d) válido y construye la matriz A_{eq} y el vector c_{obj} :

```
// Asignar índices a variables  $Y(i,d)$  válidas
int nv = 0;
int var_idx[MAX_ESTADOS][MAX_DECISIONES];
for(int i = 0; i < n; i++)
    for(int d = 0; d < m.num_decisiones; d++)
        var_idx[i][d] = m.estados_afectados[d][i] ? nv++ : -1;

// Restricción de balance para el estado destino s:
// + $Y(s,d)$  para toda  $d$  disponible en  $s$ 
// - $P(i \rightarrow s | d) \cdot Y(i,d)$  para todo par  $(i,d)$  válido
for(int i = 0; i < n; i++)
    for(int d = 0; d < m.num_decisiones; d++)
        if(var_idx[i][d] >= 0) {
            if(i == s) A_eq[nr][var_idx[i][d]] += 1.0;
            A_eq[nr][var_idx[i][d]] -= m.transiciones[d][i][s];
        }
```

9.3 Extracción de la Política a partir de la Solución

Una vez resuelto el PPL, la solución $Y^*(i,d)$ indica la política: para cada estado i , la decisión óptima es aquella d para la cual $Y^*(i,d) > 0$. En los vértices del poliedro factible (que es donde el Simplex siempre termina), exactamente una decisión por estado tendrá $Y > 0$, lo que determina una política pura determinista.

9.4 Detección de Políticas Alternativas Óptimas

El PPL puede tener múltiples óptimos cuando la función objetivo es paralela a una cara del poliedro factible. El Simplex llega a un vértice arbitrario de esa cara. Para detectar **todos** los empates, el programa realiza un barrido complementario con Gauss-Jordan sobre todas las políticas generadas, identifica las que tienen $|g - g^*| < 1e-4$, y entre las empatadas elige como política reportada la de mayor/menor $V(\text{estado}_0)$ como criterio de desempate.

10. Método 3: Mejoramiento de Políticas (Howard)

Método iterativo que parte de una política inicial π_0 y la mejora sucesivamente hasta alcanzar la política óptima. Cada iteración consta de dos pasos: evaluación de la política actual (resolver el sistema de Howard) y mejoramiento para obtener una política estrictamente mejor o igual (en cuyo caso se detiene).

10.1 Las Ecuaciones de Evaluación de Howard

Para la política actual π se buscan la tasa de ganancia/costo g y los valores relativos $V = (V_0, V_1, \dots, V_{n-2})$ que satisfacen:

$$g + V_i - \sum_j P(j|i, \pi(i)) \cdot V_j = C(i, \pi(i)) \quad \forall i$$

Este sistema tiene n ecuaciones y n incógnitas: $g, V_0, V_1, \dots, V_{n-2}$. La condición $V_{n-1} = 0$ fija la escala de V (es la normalización del sistema de Howard, análoga a $\sum \pi_i = 1$ en la enumeración exhaustiva).

```
// Construir matriz aumentada del sistema de Howard
// Fila i:  g + V_i - P[d][i][0]·V_0 - P[d][i][1]·V_1 - ... = C(i,d)
for(int i = 0; i < n; i++) {
    int d = pol[i]; // decisión actual en el estado i
    A[i][0] = 1.0; // coeficiente de g
    for(int j = 0; j < n-1; j++)
        A[i][j+1] = -m.transiciones[d][i][j]; // coeficientes de V_0..V_{n-2}
    A[i][i+1] += 1.0; // +1 en la diagonal (coeficiente de V_i)
    A[i][n] = m.costos[i][d]; // RHS
}
// Nota: la columna para V_{n-1} no aparece porque V_{n-1} = 0 (fijo)
```

10.2 Paso de Mejoramiento

Con los valores V obtenidos, para cada estado i se evalúa $Q(i,d) = C(i,d) + \sum_j P(j|i,d) \cdot V_j$ para todas las decisiones d disponibles. La nueva política elige el d que minimiza (costos) o maximiza (ganancias) $Q(i,d)$:

```
int igual = 1; // bandera de convergencia
for(int i = 0; i < n; i++) {
    double mejor = (tipo==0) ? 1e30 : -1e30;
    int best = pol[i]; // mantener la decisión actual como default
    for(int d = 0; d < m.num_decisiones; d++) {
        if(!m.estados_afectados[d][i]) continue;
        double sum = 0;
        for(int j = 0; j < n-1; j++) // V_{n-1} = 0, no se suma
            sum += m.transiciones[d][i][j] * V[j];
        double val = m.costos[i][d] + sum; // Q(i,d)
        if((tipo==0 && val<mejor)|| (tipo==1 && val>mejor))
            { mejor=val; best=d; }
    }
    if(best != pol[i]) { igual = 0; pol[i] = best; }
}
if(igual) break; // convergencia: política no cambió
```

CONVERGENCIA GARANTIZADA: El número de políticas es finito. Cada iteración produce

una política con g estrictamente menor (costos) o mayor (ganancias) que la anterior, o la misma política (convergencia). Por lo tanto, el algoritmo nunca puede ciclar y termina en a lo más $|\Pi|$ iteraciones. En la práctica converge en 3-10 iteraciones incluso para modelos con docenas de estados.

11. Método 4: Mejoramiento con Descuento

Variante del Mejoramiento de Políticas con un factor de descuento $\alpha \in (0,1)$. Este modelo refleja la preferencia por costos/ganancias presentes sobre los futuros: un valor que ocurrirá t pasos en el futuro se pondera por α^t . El valor por defecto $\alpha = 0.9$ significa que un costo 10 etapas en el futuro vale $0.9^{10} \approx 0.35$ veces un costo inmediato.

11.1 Ecuación de Bellman con Descuento

La ecuación de Bellman (o de optimalidad) para el modelo con descuento es:

$$V_i = C(i, \pi(i)) + \alpha \cdot \sum_j P(j|i, \pi(i)) \cdot V_j \quad \forall i$$

Reordenando: $V_i - \alpha \cdot \sum_j P(j|i, \pi(i)) \cdot V_j = C(i, \pi(i))$, lo que en forma matricial es $(I - \alpha P)V = c$.

```
// Construcción del sistema (I - αP)V = c
for(int i = 0; i < n; i++) {
    int d = pol[i];
    for(int j = 0; j < n; j++)
        A[i][j] = (i==j)
            ? 1.0 - alpha * m.transiciones[d][i][j] // diagonal: 1 - α·Pii
            : - alpha * m.transiciones[d][i][j]; // fuera diagonal: -α·Pij
    A[i][n] = m.costos[i][d]; // RHS = costo en estado i bajo pol actual
}
```

DIFERENCIA CLAVE entre los dos métodos de Howard: Sin descuento (método 3): el sistema tiene n incógnitas ($g, V_0, \dots, V_{n-2}$), g es la tasa de costo por etapa y los V son relativos ($V_{n-1} = 0$ fija la escala). Con descuento (método 4): el sistema es cuadrado $n \times n$, no aparece g , y los V son valores totales descontados. La solución existe y es única porque $\alpha < 1$ hace que $I - \alpha P$ sea estrictamente diagonal dominante ($|A_{ii}| = 1 - \alpha \cdot P_{ii} > 0 > \sum_{j \neq i} |A_{ij}| = \alpha \cdot \sum_{j \neq i} P_{ij}$ para matrices estocásticas), lo que garantiza invertibilidad por el criterio de Hadamard.

12. Método 5: Aproximaciones Sucesivas (Value Iteration)

También llamado iteración de valores o value iteration. No resuelve sistemas de ecuaciones lineales: actualiza el vector V directamente mediante la ecuación de optimalidad de Bellman hasta que las diferencias entre iteraciones consecutivas son menores que ε . Es el método más simple de implementar entre los iterativos.

12.1 El Algoritmo Paso a Paso

Iteración	Cálculo	Criterio de parada
Inicialización ($k=1$)	$V^1_i = \text{opt_d } C(i,d)$ según el tipo (mín para costos, máx para ganancias)	—
$k = 2, 3, 4, \dots$	$V^k_i = \text{opt_d } [C(i,d) + \alpha \cdot \sum_j P(j i,d) \cdot V^{k-1}_j]$	$\max_i V^k_i - V^{k-1}_i < \varepsilon$

La política óptima se extrae en la última iteración: para cada estado i , la decisión es aquella d que logró el valor V^k_i .

12.2 Implementación del Bucle Principal

```
// Inicialización  $V^1$ : min o max de costos directos
for(int i = 0; i < n; i++) {
    V[i] = (tipo==0) ? 1e30 : -1e30;
    for(int d = 0; d < m.num_decisiones; d++) {
        if(!m.estados_afectados[d][i]) continue;
        if((tipo==0 && m.costos[i][d] < V[i]) ||
            (tipo==1 && m.costos[i][d] > V[i]))
            V[i] = m.costos[i][d];
    }
}

// Iteraciones hasta convergencia
for(int iter = 2; iter <= max_iter; iter++) {
    double max_dif = 0;
    for(int i = 0; i < n; i++) {
        double best_q = (tipo==0) ? 1e30 : -1e30;
        for(int d = 0; d < m.num_decisiones; d++) {
            if(!m.estados_afectados[d][i]) continue;
            double q = m.costos[i][d];
            for(int j = 0; j < n; j++)
                q += alpha * m.transiciones[d][i][j] * V[j];
            if((tipo==0 && q < best_q) || (tipo==1 && q > best_q)) best_q = q;
        }
        V_nuevo[i] = best_q;
        double dif = fabs(V_nuevo[i] - V[i]);
        if(dif > max_dif) max_dif = dif;
    }
    memcpy(V, V_nuevo, n*sizeof(double));
    if(max_dif < epsilon_aproximacion) break;
}
```

NOTA SOBRE $\alpha = 1$: La convergencia teórica de Value Iteration está garantizada solo para $\alpha < 1$ (el operador de Bellman es una contracción con módulo α). Con $\alpha = 1$ la convergencia no está garantizada en general. Sin embargo, para modelos de costo promedio con cadenas de Markov ergódicas (que es el caso típico en los MDP de este programa), el algoritmo converge en la práctica porque las diferencias $V^k - V^{(k-1)}$ se estabilizan aunque los valores absolutos de V crezcan indefinidamente. El criterio de parada por diferencia máxima captura esta estabilización práctica.

13. Módulo de Comparación de Métodos

La función `comparacion()` ejecuta los cinco métodos sobre el mismo modelo con los alfas configurados y presenta una tabla resumen. Cada método corre en modo silencioso (`mostrar = 0`), guardando sólo la política óptima y el valor g^* sin imprimir los pasos intermedios.

13.1 Variables Globales de Configuración

Variable global	Valor por defecto	Afecta a
<code>alpha_descuento</code>	0.9	Mejoramiento con Descuento (método 4)
<code>alpha_aproximacion</code>	1.0	Aproximaciones Sucesivas (método 5)
<code>epsilon_aproximacion</code>	0.001	Aproximaciones Sucesivas (individual y comparación); se actualiza cuando el usuario ingresa ϵ en el método individual

Ambos alfas se configuran desde la opción 6 del menú principal: $\alpha_{\text{descuento}} \in (0,1)$ estricto (no puede ser 0 ni 1), $\alpha_{\text{aproximacion}} \in (0,1]$ inclusive el 1 (porque 1 es el valor más común para el criterio de costo promedio).

13.2 Análisis de Coincidencias y Políticas Alternativas

Al finalizar la tabla resumen, el programa realiza dos análisis adicionales:

Coincidencias entre métodos: si todos los métodos reportan la misma política, se muestra un mensaje verde de confirmación. Si difieren, se muestran grupos de métodos que coinciden entre sí y se señala si el desacuerdo puede explicarse por empates de valor (dos políticas con g^* igual pero política distinta).

Políticas alternativas globales: se hace un barrido completo de todas las políticas con Gauss-Jordan para listar todas las que tienen el mismo g^* . Esta sección aparece independientemente de si los métodos coincidieron o no.

13.3 Reporte en Archivo .txt

Al terminar la comparación se ofrece guardar un reporte .txt con: fecha y hora de generación, parámetros del modelo (estados, decisiones, tipo, alfas configurados), resultados por método (política, costo/ganancia, valores V finales para métodos 4 y 5) y la conclusión sobre coincidencia entre métodos.

14. Sistema de Archivos: Guardar y Cargar

14.1 Formato del Archivo .mdp

Los archivos .mdp son texto plano con estructura fija. La extensión es el acrónimo de Markov Decision Process. El formato permite cargar exactamente el mismo modelo en otra sesión sin reingresar datos:

```

tipo      num_estados  num_decisiones  <- línea 1: tres enteros
nombre_estado_0  <- un nombre por línea
...            <- (num_estados líneas)
nombre_decision_0
...            <- (num_decisiones líneas)

// Para cada decisión d (0 a k-1):
af[0] af[1] ... af[n-1]      <- 1 ó 0, sep. por espacios
costo[0] costo[1] ... costo[n-1]  <- double, 10 decimales
// n filas de transición:
P[d][0][0] P[d][0][1] ... P[d][0][n-1]
P[d][1][0] ...
...
```

14.2 ¿Por qué guardar con 10 decimales?

El tipo `double` tiene aproximadamente 15-16 dígitos significativos. Al guardar con `%.10f` se preservan 10 decimales, suficiente para que al recargar el modelo las probabilidades sean idénticas a las originales dentro de la precisión del tipo `double`.

Con menos decimales (por ejemplo 4), una probabilidad exacta como $1/3 = 0.333333\dots$ se guardaría como 0.3333 y al recargar la suma de la fila sería 0.9999 en lugar de 1.0, disparando innecesariamente la advertencia de suma $\neq 1$.

15. Sistema de Validaciones

15.1 Validación de Entradas Numéricas

Situación	Mecanismo de protección
Usuario escribe letras o texto	strtod() retorna end==buf (puntero no avanzó); se pide reintentar
Entrada vacía (solo Enter)	buf[0]=='\0' detectado; se pide reintentar con mensaje informativo
División entre cero en fracción a/b	fabs(d)<1e-9 detectado antes de dividir; se pide reintentar
Entero fuera del rango [min,max]	leer_entero_rango() verifica y reintenta en bucle infinito
Probabilidad individual fuera de [0,1]	Advertencia en rojo y opción de reingresar la fila completa (hasta 3 intentos)
Suma de fila de transición $\neq$ 1.0	Advertencia con la suma obtenida (tolerancia 0.001); hasta 3 intentos de reingresar
Costos/ganancias incorrectos	Verificación interactiva post-ingreso: resumen numerado y corrección por ítem

15.2 Validación del Estado del Modelo

La variable global `bool modelo_cargado` actúa como guardia de seguridad. Las opciones 2 (Visualización), 3 (Métodos), 4 (Comparación) y Guardar del menú verifican este flag antes de ejecutar:

```
case 2:
    if(!modelo_cargado) {
        printf("\033[1;31mPrimero ingresa o carga un modelo.\033[0m\n");
        pausar();
    } else visualizar();
    break;
```

La variable se pone en `true` al final de `ingresar_datos()` y también al cargar exitosamente un archivo `.mdp` con `cargar_modelo()`.

16. Interfaz de Terminal

16.1 Códigos de Color ANSI

Código	Efecto	Uso en el programa
\033[1;33m	Amarillo negrita	Encuadre del menú principal; títulos de decisiones en ingreso
\033[1;34m	Azul negrita	Títulos de secciones (== INGRESO DE DATOS ==)
\033[1;32m	Verde negrita	Resultados óptimos, confirmaciones, modelo cargado
\033[1;31m	Rojo negrita	Errores, advertencias, modelo no cargado
\033[1;37m	Blanco negrita	Texto destacado en la animación de despedida
\033[0m	Reset	Volver al color por defecto del terminal tras cada bloque coloreado

Los códigos ANSI son interpretados por la mayoría de terminales en Linux, macOS y Windows 10+. En terminales que no los soporten, se verán como secuencias de caracteres literales. El programa no detecta si el terminal soporta ANSI; se asume que sí.

16.2 Animación de Despedida

Fase	Técnica	Duración aprox.
1. Créditos	printf con colores y bordes Unicode (̐ ̑ ̒ ̓ =)	Instantáneo
2. Barra de puntos	printf('.') + fflush(stdout) en bucle de 20 × 80 ms	~1.6 segundos
3. Fuegos artificiales	\033[1A (subir línea) + \033[2K (borrar línea) para reanimar frame a frame	~1.5 segundos

¿Por qué fflush(stdout) es esencial? Sin él el sistema operativo acumula el output en el buffer de E/S y lo muestra todo de golpe al final, destruyendo el efecto de animación progresiva. fflush fuerza el vaciado inmediato del buffer tras cada carácter o frame.

17. Guía de Preguntas Frecuentes en Presentación

Esta sección prepara respuestas concretas y detalladas a preguntas técnicas que podría plantear la profesora durante la revisión del proyecto.

¿Por qué se usa Gauss-Jordan y no Gauss simple?

Gauss simple sólo elimina hacia abajo (triangulación) y requiere un paso adicional de sustitución hacia atrás para obtener la solución. Gauss-Jordan elimina en ambas direcciones, obteniendo la forma identidad reducida directamente. La solución se lee de la última columna $x_i = a[i][n]$ sin ningún cálculo adicional. La función queda autocontenida: quien la llama pasa la matriz aumentada y lee la solución en esa misma columna al regresar.

¿Por qué el Simplex de Dos Fases y no el Método de la Gran M?

El Método de la Gran M asigna un coeficiente enorme M a las variables artificiales en la función objetivo original. Con $M = 10^6$ y costos en el orden de las centenas, las operaciones de punto flotante acumulan errores que pueden hacer que el algoritmo tome decisiones incorrectas de pivoteo. Las Dos Fases evitan completamente este problema usando dos funciones objetivo separadas: primero minimizar la suma de artificiales (coeficientes exactamente 1), luego optimizar el objetivo real sin artificiales.

¿Cuál es la diferencia fundamental entre Mejoramiento con y sin descuento?

Sin descuento (método 3): se minimiza el costo promedio por etapa en horizonte infinito. El sistema de Howard tiene n incógnitas $(g, V_0, \dots, V_{n-2})$; g es la tasa de costo y los V son relativos, con $V_{n-1} = 0$ como normalización.

Con descuento (método 4): se minimiza el valor presente total $\sum_t \alpha^t \cdot C$. El sistema $(I - \alpha P)V = c$ es cuadrado $n \times n$ con solución única garantizada porque $\alpha < 1$ hace que $I - \alpha P$ sea estrictamente diagonal dominante (criterio de Hadamard). Los V son valores totales descontados, no tasas por etapa.

¿Por qué Programación Lineal puede tener múltiples políticas óptimas?

El conjunto factible del PPL es un poliedro convexo en $\mathbb{R}^n$. Cuando la función objetivo es paralela a una cara del poliedro, todos los puntos de esa cara son igualmente óptimos. El Simplex termina en un vértice arbitrario de esa cara. Para detectar todos los empates el programa realiza un barrido explícito con Gauss-Jordan sobre todas las políticas generadas, lo que identifica los demás vértices óptimos.

¿Por qué Aproximaciones Sucesivas puede usar $\alpha = 1$?

La convergencia teórica de Value Iteration está garantizada sólo para $\alpha < 1$ (contracción). Sin embargo, para modelos de costo promedio con cadenas de Markov ergódicas, el algoritmo converge en la práctica con $\alpha = 1$ porque las diferencias $V^k - V^{k-1}$ se estabilizan aunque los valores absolutos de V crezcan indefinidamente. El criterio de parada en el programa (diferencia máxima $< \epsilon$) captura esta estabilización práctica.

¿Por qué los buffers `g_politicas` y `g_optimas_pol` son globales?

Con `MAX_POLITICAS = 100 000` y `MAX_ESTADOS = 20`, cada arreglo bidimensional ocupa $100\,000 \times 20 \times 4 \text{ bytes} = 8 \text{ MB}$. Declararlos como variables locales (en el stack) desbordaría el stack del proceso en la mayoría de sistemas operativos (que limitan el stack a 1-8 MB). Al declararlos `static` a nivel de archivo, el compilador los coloca en el segmento BSS/data del proceso, que no tiene esa restricción.

¿Por qué guardar con 10 decimales en el archivo `.mdp`?

El tipo `double` tiene ~15 dígitos significativos. Guardar con `%.10f` asegura que al recargar el modelo, probabilidades como `1/3` mantengan exactamente el mismo valor que tenían. Con menos decimales, la suma de la fila al recargar puede diferir de 1.0 en la última cifra significativa guardada, disparando innecesariamente la advertencia de validación.

¿Por qué se usa `goto` en la validación de matrices?

En C, `goto` tiene mala fama por el artículo de Dijkstra "Go To Statement Considered Harmful" (1968), pero ese artículo critica el `goto` arbitrario que dificulta el análisis del flujo de control. El `goto retry_trans` del programa salta hacia **atrás** a un punto claramente etiquetado en el mismo bloque de lectura: es equivalente a un `redo` (que C no tiene). Una alternativa con `do-while` requeriría una bandera booleana adicional y dos niveles más de indentación sin ningún beneficio de claridad.

18. Resumen Comparativo de los Cinco Métodos

Método	Tipo	Ecuación central	α	Estructura de datos
Enumeración Exhaustiva	Exacto	$\pi P = \pi, \sum \pi = 1$ → Gauss-Jordan	No	<code>g_politicas[]</code> global
Programación Lineal	Exacto	Max/Min $\sum C \cdot Y$ s.a. balance de flujo → Simplex	No	Tabla Simplex dinámica
Mej. Políticas	Iterativo	$g + V = C + PV$ → Gauss-Jordan	No	Vectores <code>pol[n]</code> , <code>V[n-1]</code>
Mej. con Descuento	Iterativo	$(I - \alpha P)V = c$ → Gauss-Jordan	(0,1)	Matriz <code>A[n][n+1]</code>
Aprox. Sucesivas	Iterativo	$V^k = \text{opt}(C + \alpha \cdot PV^{k-1})$ directo	(0,1]	Vectores <code>V[n]</code> , <code>V_nuevo[n]</code>

Método	Garantía de óptimo	Detección de empates	Complejidad
Enumeración Exhaustiva	Sí (exhaustivo)	Sí, lista todas	$O(\Pi \cdot n^3)$
Programación Lineal	Sí (PPL convexo)	Sí, barrido complementario	$O(\text{Simplex}) \approx O(n^2 \cdot nv)$
Mej. Políticas	Sí (finito, monótono)	No (reporta una)	$O(\text{iter} \cdot n^3)$
Mej. con Descuento	Sí (contracción $\alpha < 1$)	No (reporta una)	$O(\text{iter} \cdot n^3)$
Aprox. Sucesivas	Sí ($\alpha < 1$); práctico $\alpha = 1$	No (reporta una)	$O(\text{iter} \cdot n^2 \cdot D)$